

Spécification et preuves de programme pour n°2 - ZANON N°51F

1) Table de vérité

L'objectif est de vérifier que l'opérateur $\&\&$ se comporte exactement comme la conjonction logique. On utilise un lemme universel.

lemma verify_and:

forall b1 b2: bool

(b1 = true $\wedge$ b2 = true $\leftrightarrow$ (b1 $\&\&$ b2) = true) $\wedge$

(b1 = false $\vee$ b2 = false $\leftrightarrow$ (b1 $\&\&$ b2) = false)

Ce lemme couvre les 4 cas possibles.

2) Recherche entre deux tableaux

1) La fonction search effectue une recherche linéaire. Elle est spécifiée par renvoyer soit un indice valide, soit -1. Donc retourner soit le premier occurrence de x soit -1 si absent.

2.3)

let search (a: array int) (x: int): int

ensures { result = -1 $\leftrightarrow$ forall k. $0 \leq k < a.length \rightarrow a[k] \neq x$ }

ensures { result $\neq$ -1 $\rightarrow 0 \leq result < a.length \wedge a[result] = x$ }

=

let ref i := 0 in

let ref index := -1 in

while i < a.length & index = -1 do

variant { a.length - i }

invariant { $0 \leq i < a.length$ }

invariant { index = -1 $\rightarrow$ forall k. $0 \leq k < i \rightarrow a[k] \neq x$ }

invariant { index $\neq$ -1 $\rightarrow 0 \leq index < a.length \wedge a[index] = x$ }

if a[i] = x then i := i;

i := i + 1

done;

index

4) La fonction minus copie les éléments de A qui ne sont pas dans B vers C. Voici un exemple qui respecte la condition

a = [5, 8, 3, 10]

b = [8, 10, 1, 2]

resultat c = [5, 3] la fonction retourne.

2. (5, 6, 7 et 8) let diff (a b c: array int): int
 requires { c.length ≥ a.length + b.length }
 ensures { 0 ≤ result ≤ a.length + b.length }
 =
 let ref i = 0 in
 for j = 0 to a.length - 1 do
 invariant { 0 ≤ i ≤ j }
 if search b a[j] = -1 then begin
 c[i] ← a[j];
 i := i + 1
 end
 done
 let ref m = i in
 for j = 0 to b.length - 1 do
 invariant { i ≤ m ≤ i + j }
 if search a b[j] = -1 then begin
 c[m] ← b[j];
 m := m + 1
 end
 done;
 m

3) longueur d'une concaténation de deux listes

1. type inductif des listes

type list 'a =

| Nil

| Cons 'a (list 'a)

Fonction length

let rec fonction length (l: list 'a): int =

match l with

| Nil → 0

| Cons . r → 1 + length r

end

Cela calcule la longueur d'une liste

Fonction append

```
let rec function append (l m: list 'a) : list 'a =  
  match l with  
  | Nil → m  
  | Cons x r → Cons x (append r m)  
end
```

Concatène deux listes

3.2) On veut exprimer $\text{length}(\text{append}(l, m)) = \text{length}(l) + \text{length}(m)$

goal length-append :

forall l m : list 'a.

$\text{length}(\text{append } l \text{ } m) = \text{length } l + \text{length } m$

3) Les solveurs de preuves automatiques ne savent pas faire d'induction d'eux-même, il ne deviennent pas qu'il faut faire une induction sur l .

4) on ajoute une preuve par induction sous forme de fonction récursive.

```
let rec lemma length-append-lemma (l m: list 'a) : unit  
  variant {l}  
  ensures {length (append l m) = length l + length m}  
= match l with  
  | Nil → ()  
  | Cons x r → length-append-lemma r m  
end
```

5) On encode directement la propriété par une post condition.

```
let rec function append (l m: list 'a) : list 'a  
  ensures {length result = length l + length m}  
= match l with  
  | Nil → m  
  | Cons x r → Cons x (append r m)  
end
```